

QR Code Follow (No-PTZ)

1. Experiment Objective

The camera tracks a QR code, with body rotation tracking + distance following keeping the QR code centered in the frame and maintaining a constant distance.

2. Experiment Principle

1. Key Terms

Term	Description
Visual Follow	Adds distance control (linear.x) on top of tracking, making the car rotate and move forward/backward simultaneously
Distance Follow	Estimates distance from the QR code pixel size, with PD control keeping the car at the desired distance
pyzbar Decoding	Decodes the QR code content based on the open-source library pyzbar, displayed in real time on the terminal
Stable-frame Gating	Only confirms following after the same target is detected for <code>follow_stable_frames</code> (3) consecutive frames
QoS	best_effort (image input + cmd_vel downlink), depth=1

2. Technical Architecture

Shares the node `qr_tracker_node`, with `enable_distance_follow:=true`:

```
/camera/image_raw (BEST_EFFORT)
└→ qr_tracker_node (enable_distance_follow:=true)
    ├── pyzbar/OpenCV dual-backend QR detection → stable-frame gating
    └── PD angular velocity + QR code size distance following → /cmd_vel
```

3. Core Topics Quick Reference

Firmware Downlink

Topic	Message Type	QoS	Field	Range	Direction
<code>/cmd_vel</code>	<code>geometry_msgs/Twist</code>	best_effort + keep_last(1)	<code>linear.x</code>	-0.18~0.18 m/s	<code>+=</code> forward, <code>-=</code> backward
			<code>angular.z</code>	-0.60~0.60 rad/s	<code>+=</code> left turn, <code>-=</code> right turn

3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ Version	✗
No-PTZ Version	☑
Required Peripherals	ESP32 camera (mounted on bracket), a printed or phone-displayed QR code

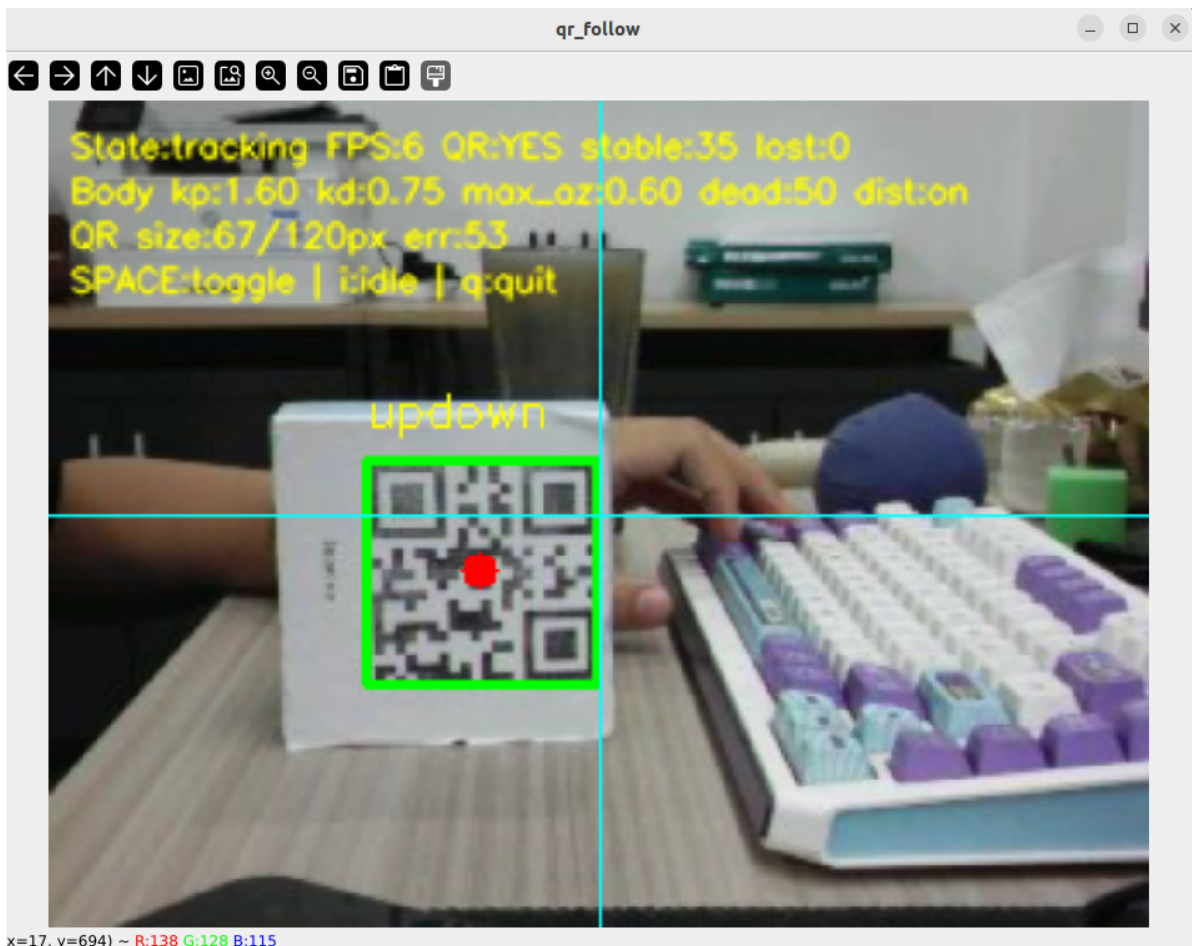
2. Start Agent + Camera + Joint Model (Terminal 1)

```
ros2 launch microros_v2_bringup car_base.launch.py
```

```
yahboon@yahboon:~$ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboon/.ros/log/2026-07-28-15-04-46-269642-yahboon-579370
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [579411]
[INFO] [camera_driver-2]: process started with pid [579413]
[INFO] [robot_state_publisher-3]: process started with pid [579415]
[INFO] [joint_state_publisher-4]: process started with pid [579417]
[INFO] [ekf_node-5]: process started with pid [579419]
[micro_ros_agent-1] [1785222287.404236] info      | UDPv4AgentLinux.cpp | init                | running...         | port: 8090
[robot_state_publisher-3] [INFO] [1785222288.847816851] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785222288.848013042] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785222288.848028942] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848037122] [robot_state_publisher]: got segment cam1_Link
[robot_state_publisher-3] [INFO] [1785222288.848043874] [robot_state_publisher]: got segment cam2_Link
[robot_state_publisher-3] [INFO] [1785222288.848050538] [robot_state_publisher]: got segment cam3_Link
[robot_state_publisher-3] [INFO] [1785222288.848057264] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785222288.848063747] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785222288.848070240] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848076888] [robot_state_publisher]: got segment rfwheel_Link
[joint_state_publisher-4] [INFO] [1785222289.372349515] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description topic..
[camera_driver-2] [INFO] [1785222289.716161193] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[camera_driver-2] [WARN] [1785222292.705741414] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785222294.072701944] [esp32_ip_camera_publisher]: IP camera stream opened successfully
```

3. Start QR Code Follow (Terminal 2)

```
ros2 launch microros_v2_no_ptz_visual_control_pkg qr_follow.launch.py
```



5. Operation Instructions

Key	Function
Space	Toggle follow/standby
q / ESC	Exit

4. Experiment Result

- QR code approaches → the car moves backward to keep the distance
- QR code moves away → the car moves forward to follow
- The terminal outputs the decoded QR code content in real time
- Stable-frame gating prevents false triggering
- Auto-stops after target-loss timeout

5. Code Explanation

Source code paths:

- Launch:


```
~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/launch/qr_follow.launch.py
```
- Node source:


```
~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/qr_tracker_node.py
```

1. Core Node Explanation

`qr_follow.launch.py` launches `qr_tracker_node`, with `enable_distance_follow` defaulting to `true`. Distance following computes the PD linear velocity from the deviation between the QR code bounding rectangle size and the desired size `target_qr_size_px` (120 px). Stable-frame gating (detected for 3 consecutive frames) prevents occasional false detections from triggering following.

2. Key Parameters

Parameter definition files:

- Launch:
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/launch/qr_follow.launch.py`
- Node:
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/qr_tracker_node.py`

Parameter	Type	Default	Description
<code>qr_detector_backend</code>	str	<code>pyzbar</code>	QR decoding backend
<code>follow_stable_frames</code>	int	<code>3</code>	Stable-frame gating frame count
<code>deadband_x_pixel</code>	float	<code>50.0 px</code>	Horizontal deadband
<code>angular_kp</code> / <code>angular_kd</code>	float	<code>1.6 / 0.75</code>	Angle PD gains
<code>max_angular_z</code>	float	<code>0.6 rad/s</code>	Maximum turning speed
<code>enable_distance_follow</code>	bool	<code>true</code>	Enable distance following
<code>target_qr_size_px</code>	float	<code>120.0 px</code>	Desired QR code pixel size
<code>distance_tolerance_qr_size_px</code>	float	<code>20.0 px</code>	Distance tolerance
<code>distance_kp</code>	float	<code>0.05</code>	Distance PD proportional gain
<code>max_linear_x</code>	float	<code>0.18 m/s</code>	Maximum forward/backward speed

3. Node Description

Node	Package	Description
<code>qr_tracker_node</code>	<code>microros_v2_no_ptz_visual_control_pkg</code>	QR code tracking + following, stable-frame gating + dual PD

6. Common Issues

Issue	Cause	Solution
Frequent target loss	QR code too small or moving too fast	Enlarge the QR code and slow down the movement
Inaccurate distance following	Unreasonable distance PD parameters	Adjust <code>distance_kp</code> / <code>target_qr_size_px</code>
Decoding failure	QR code blurry or reflective	Make sure the QR code is clear and flat